

# RULE-BASED CONVERSATIONAL CHATBOT

Student: K. Niharika    RegNo: 99230040633

## 1. Objective

The objective is to build a simple rule-based conversational chatbot in Python that can interact continuously with a user, recognize different types of messages, provide suitable responses, and terminate the conversation when requested. The implementation follows the required flow: INPUT → PROCESS → RESPOND → REPEAT → EXIT.

## 2. Concepts Implemented

The chatbot uses `input()` to receive messages, `.lower().strip()` to normalize user input, `while True` for continuous interaction, and `if / elif / else` for decision-making. The `in` operator makes greeting recognition flexible, while `break` cleanly exits the loop. The user's name is also incorporated into the conversation.

## 3. Complete Python Program

```
# Rule-Based Conversational Chatbot
# 30-Minute Student Task

print("=== Welcome to the Student Chatbot ===")

name = input("What is your name? ").strip()

if not name:
    name = "Friend"

print(f"Nice to meet you, {name}!")
print("You can talk to me about Python, studies, or my purpose.")
print("Type 'bye' or 'exit' whenever you want to end the conversation.\n")

while True:
    user_input = input(f"{name}: ").lower().strip()

    if user_input in ("bye", "exit"):
        print(f"Bot: Goodbye, {name}! Have a great day.")
        break

    elif "hello" in user_input or "hi" in user_input:
        print(f"Bot: Hello, {name}! How can I help you today?")

    elif "python" in user_input:
        print(
            "Bot: Python is a high-level, general-purpose programming language. "
            "It is popular because its syntax is readable and it supports "
            "web development, automation, data analysis, AI, and more."
        )

    elif "study" in user_input or "studies" in user_input:
        print(
            "Bot: A practical study approach is to set a clear goal, "
            "break the topic into smaller sections, practice actively, "
            "and review your mistakes instead of only rereading notes."
```

```
)

elif "what can you do" in user_input or "your purpose" in user_input:
    print(
        "Bot: I am a rule-based chatbot. I match keywords and phrases "
        "in your message and select a suitable response using if, "
        "elif, and else statements."
    )

elif "learn" in user_input:
    print(
        "Bot: The best way to learn programming is to write small programs, "
        "test different inputs, observe errors, and improve the code step by step."
    )

else:
    print(
        "Bot: I don't have a rule for that message yet. "
        "Try asking about Python, studies, what I can do, or say hello."
    )
```

## 4. Explanation of the Logic

First, the program welcomes the user and asks for a name before starting the main conversation. The name is cleaned using `strip()`, and a default value is provided if the user leaves it blank.

The main conversation is placed inside `while True`. Every iteration accepts a new message and normalizes it using `lower().strip()`. This means inputs such as `HELLO` and `hello` are treated consistently.

The first condition checks for `bye` or `exit`. If either is entered, the chatbot prints a goodbye message and uses `break` to terminate the loop.

The greeting condition uses the `in` operator. Therefore, messages such as `hello bot` or `hey hello` can still trigger the greeting response. This makes the chatbot more flexible without requiring complicated natural-language processing.

Additional `elif` branches handle topics such as `Python`, `studies`, the chatbot's purpose, and `learning`. If none of the rules match, the final `else` provides a useful fallback instead of leaving the user without a response.

## 5. Requirement Checklist

1. Asks for the user's name before the conversation.
2. Uses `while True` for continuous interaction.
3. Uses `input()` to receive messages.
4. Cleans input with `lower().strip()`.
5. Recognizes at least two greetings: `hello` and `hi`.
6. Answers multiple questions/topics: `Python`, `studies`, `purpose`, and `learning`.
7. Uses `if / elif / else` for decision making.
8. Uses `break` when the user types `bye` or `exit`.
9. Uses the user's name in responses.
10. Provides a useful response for unknown messages.

## 6. Sample Demonstration

What is your name? Niharika  
Nice to meet you, Niharika!

Niharika: HELLO  
Bot: Hello, Niharika! How can I help you today?

Niharika: tell me about python  
Bot: Python is a high-level, general-purpose programming language...

Niharika: how should I study  
Bot: A practical study approach is to set a clear goal...

Niharika: what can you do  
Bot: I am a rule-based chatbot. I match keywords and phrases...

Niharika: tell me about football  
Bot: I don't have a rule for that message yet...

Niharika: bye

Bot: Goodbye, Niharika! Have a great day.

## **7. Conclusion**

This implementation demonstrates the fundamentals of rule-based conversational programming. Instead of attempting complex AI or natural-language processing, it uses clearly defined rules to map user input to appropriate responses. The design is simple, readable, testable, and easy to extend by adding new conditions and responses.